

Panduan Koneksi AI Second Brain

Sambungkan tools utamamu, satu per satu: Google Drive · Email · Slack · Jira

Selamat, second brain kamu sudah hidup. Sekarang kita kasih dia **tangan**: kemampuan membaca dan menulis ke tools yang kamu pakai tiap hari. Ikuti panduan ini pelan-pelan, dari atas ke bawah. Tiap bagian berdiri sendiri, jadi kamu boleh pasang hanya yang kamu butuh dan lewati sisanya.

Panduan ini untuk kamu yang **sudah menyelesaikan Level 0** (second brain percakapan sudah jalan, `claude` sudah bisa dipanggil dari dalam folder repo). Kalau belum, selesaikan dulu `docs/INSTALL_ID.md` di dalam repo, atau baca dokumen "**Mulai Dari Sini**" yang kami kirim bareng panduan ini.

Cara Membaca Panduan Ini

Sepanjang panduan kamu akan ketemu tiga jenis blok. Kenali dulu biar gampang diikuti:

Gambar panduan (mockup layar): ilustrasi layar yang kamu tuju, dengan langkah bernomor. Ini mockup, bukan foto layar asli: tampilan aslinya bisa sedikit beda (Google & Slack sering ganti tampilan), sedangkan nama tombol dan urutannya sama.



1 Angka di gambar = urutan klik. Penjelasan nya selalu di bawah gambar, seperti baris ini.

Slot screenshot: di beberapa langkah tersulit, kami sisakan tempat kosong untuk kamu tempel **foto layar aslimu**. Ini membantu kalau kamu mengulang setup nanti,

atau membantu teman yang ikut. (Di versi Google Doc, klik slotnya lalu Insert > Image.)

SLOT SCREENSHOT: contoh

Tempel di sini: foto layar milikmu untuk langkah ini.

Callout: catatan penting di sepanjang jalan:

PENTING. Hal yang kalau dilewati bikin macet.

TIPS. Cara lebih cepat atau lebih aman.

CEK BERHASIL. Tanda bahwa langkah tadi sukses.

Aturan Emas: Kalau Macet, Tanya Claude

Sebelum mulai, satu prinsip yang bikin seluruh setup ini jauh lebih gampang dari kelihatannya:

Setiap kali ada error, jangan panik dan jangan Google-in dulu. Copy pesan errornya, tempel ke terminal `claude`, dan minta dia benerin.

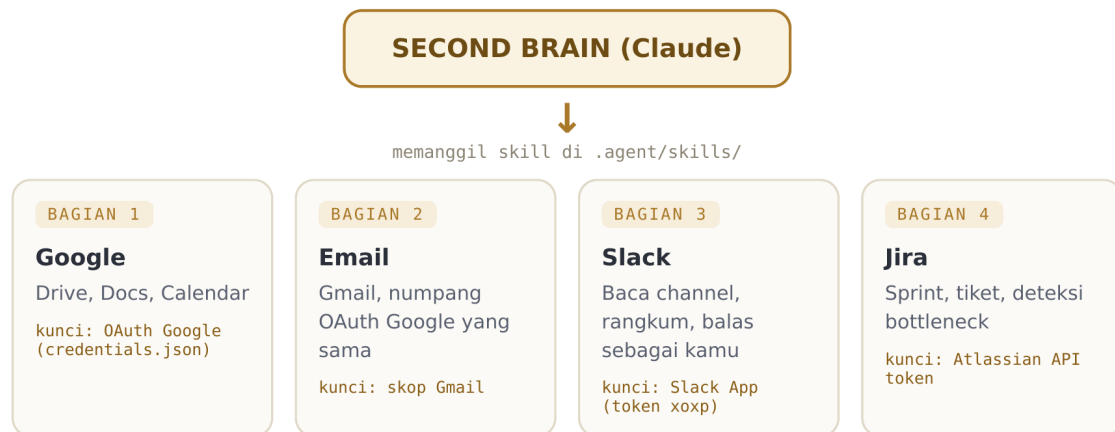
Aku dapat error ini waktu menyambungkan Google Drive. Tolong diagnosa dan perbaiki, jelaskan pelan-pelan:

[tempel pesan error lengkap di sini]

Ini lebih dari sekadar jalan pintas. Inilah inti dari jadi **AI-Native**: kamu memperlakukan AI sebagai partner yang ikut membereskan setup-nya sendiri. Setiap kali kamu lempar error ke Claude dan dia beresin, itu momen AI-Native kamu.

Peta Koneksi

Empat tools utama, dan cara mereka nyambung ke second brain kamu:



Google adalah fondasinya (Bagian 1). Email numpang di OAuth yang sama (Bagian 2). Slack dan Jira berdiri sendiri (Bagian 3 & 4). Pasang hanya yang kamu pakai.

Google adalah fondasinya (Bagian 1). Email numpang di OAuth Google yang sama (Bagian 2). Slack dan Jira berdiri sendiri (Bagian 3 & 4). Pasang sesuai kebutuhan; kamu tidak wajib pasang semuanya.

TIPS. Cara kerja "kunci": tiap skill menyimpan kredensialnya di file sendiri di dalam foldernya (`credentials.json` untuk Google, `token.env` untuk yang lain). Kunci ini **tidak pernah** ikut ter-upload ke GitHub (sudah diblokir `.gitignore`). Aman.

Bagian 1: Google Drive, Docs & Calendar

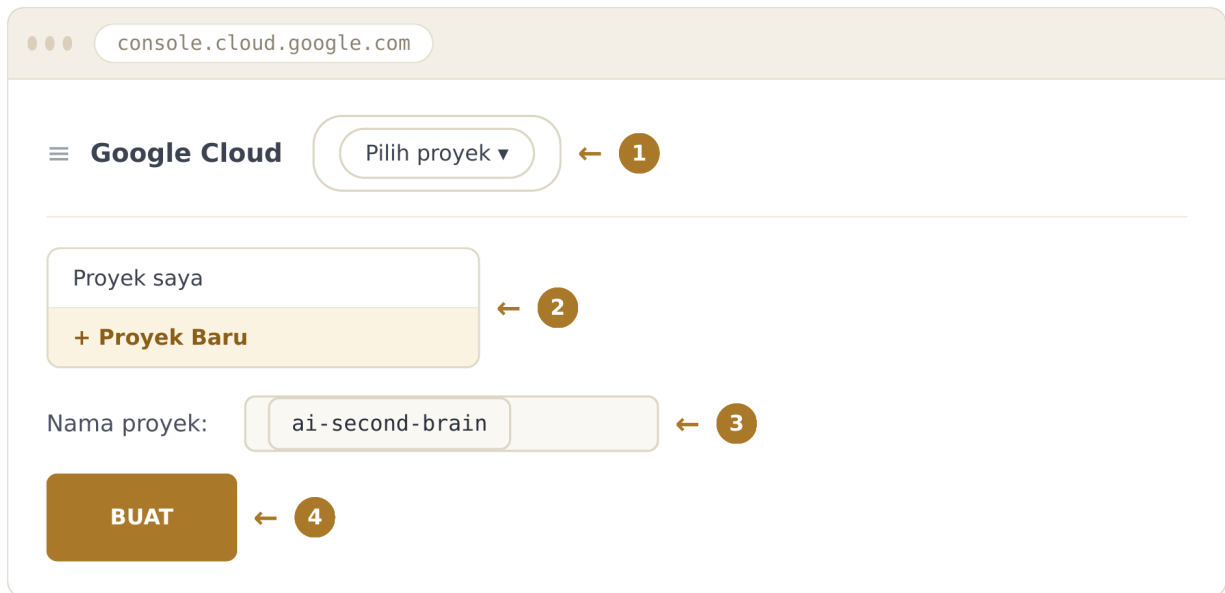
Ini bagian paling panjang dan paling teknis, tapi hanya sekali seumur setup. Setelah ini, second brain kamu bisa membaca Drive, membuat Google Docs asli, dan membaca kalendermu. Email (Gmail) numpang di setup yang sama, kita lanjutkan di Bagian 2.

Yang akan kita lakukan: bikin "proyek" di Google Cloud, nyalakan API yang dibutuhkan, lalu bikin satu **kunci OAuth** dan menaruhnya di folder connector. Sabar di bagian ini; sisanya jauh lebih cepat.

Perkiraan waktu: **20-40 menit**.

Langkah 1.1: Buat Proyek Google Cloud

Buka console.cloud.google.com dan login dengan akun Google yang mau kamu sambungkan (akun kerja atau pribadi, terserah kamu).

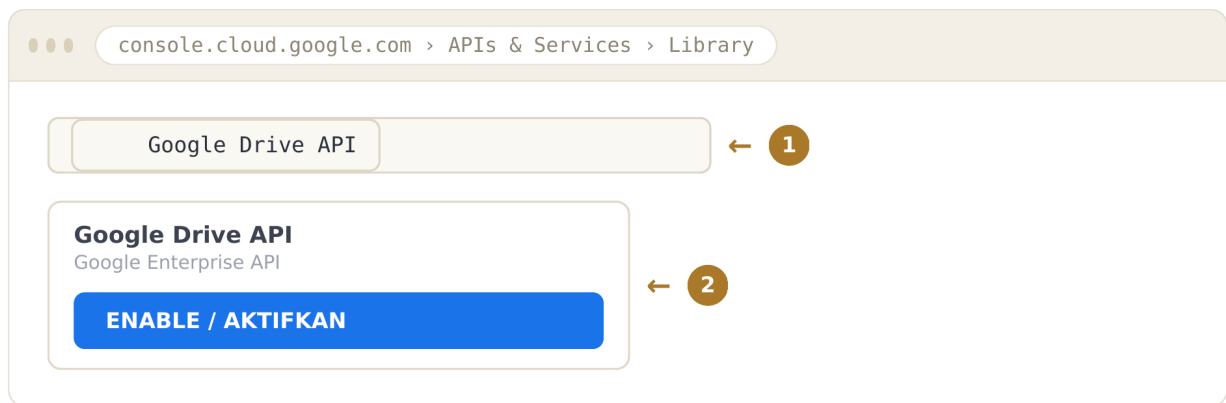


- 1 Klik dropdown pemilih proyek di bar atas.
- 2 Pilih **Proyek Baru** / **New Project**.
- 3 Beri nama yang gampang kamu kenali, misal **ai-second-brain**.
- 4 Klik **Buat**, lalu pastikan proyek baru itu terpilih di dropdown.

Langkah 1.2: Nyalakan API yang Dibutuhkan

Di menu kiri, buka **APIs & Services** → **Library**. Cari dan nyalakan (klik nama API → tombol **Enable**) satu per satu:

API	Untuk apa	Wajib?
Google Drive API	Baca & tulis file di Drive	Ya
Google Docs API	Bikin Google Docs asli	Ya
Google Sheets API	Baca & tulis spreadsheet	Disarankan
Google Calendar API	Baca kalendermu	Disarankan
Gmail API	Akses email (lihat Bagian 2)	Kalau mau email



- 1 Ketik nama API di kotak pencarian, lalu klik kartu API-nya.
- 2 Klik **Enable**. Ulangi untuk tiap API di tabel atas (Drive, Docs, Sheets, Calendar, Gmail).

TIPS. Malas nyari satu-satu? Tempel ini ke Claude: "Aku sedang setup Google OAuth untuk ai-second-brain. Sebutkan lagi 5 API yang harus aku enable di Google Cloud Console dan urutan langkahnya." Claude jadi pendamping setup-mu.

Langkah 1.3: Atur OAuth Consent Screen

Ini menentukan "izin" yang diminta aplikasimu. Buka **APIs & Services** → **OAuth consent screen**.

console.cloud.google.com > OAuth consent screen

User Type

() Internal (●) **External** ← 1

CREATE / BUAT ← 2

App name: AI Second Brain

Support email: email-mu ← 3

Developer contact: email-mu

SAVE AND CONTINUE ← 4

Test users

+ Add Users email-mu ← 5

- 1 Pilih **External**.
- 2 Klik **Create**.
- 3 Isi **App name** + email-mu di dua kolom email.
- 4 **Save and Continue**. Di layar "Scopes" langsung Save and Continue lagi.
- 5 Di layar **Test users**: klik **+ Add Users**, masukkan emailmu sendiri. Kalau dilewati, login nanti DITOLAK.

Lanjut ke layar **Test users**: klik **+ Add Users**, masukkan **alamat emailmu sendiri**, lalu **Save and Continue** → **Back to Dashboard**.

PENTING. Kalau emailmu tidak ditambahkan sebagai test user, nanti waktu login kamu akan ditolak. Jangan lewati langkah ini.

SLOT SCREENSHOT #1: OAuth Consent Screen

Tempel di sini: foto layar saat kamu memilih **External** lalu klik **Create**, dan layar **Test users** dengan emailmu sudah masuk. Ini salah satu langkah yang paling sering bikin bingung, jadi foto asli sangat membantu untuk pengulangan nanti.

Langkah 1.4: Bikin Kunci OAuth (Credentials)

Buka **APIs & Services** → **Credentials**.

console.cloud.google.com > APIs & Services > Credentials

+ CREATE CREDENTIALS ▾

API key

OAuth client ID

Service account

Application type: Desktop app ▾

Name: AI Second Brain Desktop

CREATE

✓ OAuth client created

DOWNLOAD JSON

- 1 Klik + **Create Credentials**.
- 2 Pilih **OAuth client ID**.
- 3 Application type = **Desktop app**. WAJIB Desktop, bukan Web, kalau salah login nanti gagal.
- 4 Kasih nama, klik **Create**.
- 5 Di popup, klik **Download JSON**, lalu ubah namanya jadi persis **credentials.json**.

Ubah nama file yang terunduh menjadi persis **credentials.json**.

PENTING. Application type harus **Desktop app**. Kalau kamu salah pilih "Web application", proses login nanti akan gagal.

SLOT SCREENSHOT #2: Download credentials.json

Tempel di sini: foto layar popup "**OAuth client created**" dengan tombol **Download JSON**.

Langkah 1.5: Taruh credentials.json di Folder Connector

Salin `credentials.json` ke folder connector Google. Jalankan dari dalam folder repo (di terminal `claude` atau terminal VS Code):

```
# Untuk akun kerja utama:
cp ~/Downloads/credentials.json .agent/skills/work-drive-connector/credentials.json

# (Opsional) untuk akun Google pribadi terpisah:
cp ~/Downloads/credentials.json .agent/skills/personal-drive-connector/credentials.json
```

TIPS. Gak yakin lokasi file atau perintah copy-nya? Tempel ke Claude: "File credentials.json ada di folder Downloads-ku. Tolong salin ke .agent/skills/work-drive-connector/credentials.json." Dia yang jalanin.

Langkah 1.6: Login Pertama Kali

Jalankan perintah ini. Perintah "search" ini sengaja dipakai untuk memicu proses login:

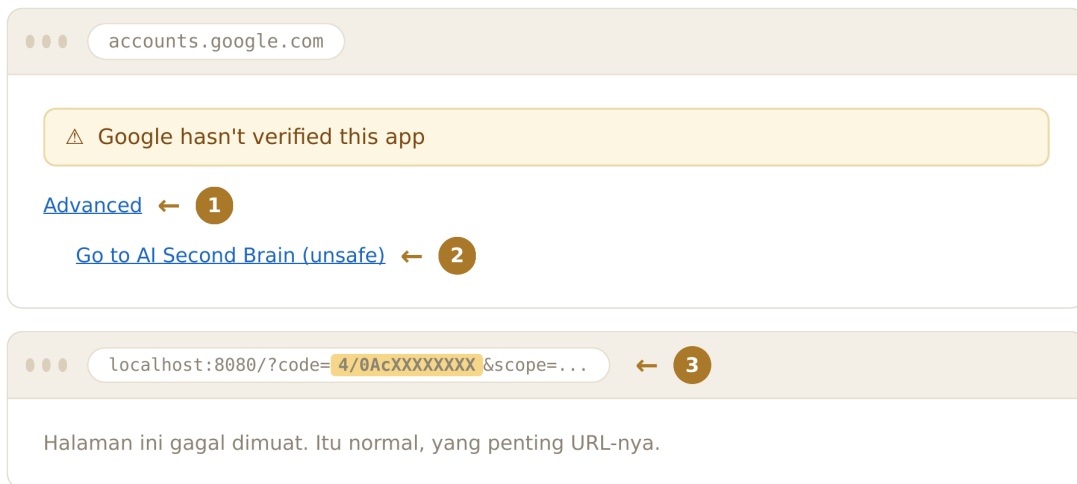
```
python3 .agent/skills/work-drive-connector/gdrive_manager.py search --query "test"
```

Kamu akan lihat output seperti ini di terminal:

[Work Drive] Authentication Required!

1. Buka URL ini di browser:
`https://accounts.google.com/o/oauth2/v2/auth?...`
2. Izinkan aplikasi. Kalau muncul "Google hasn't verified this app", klik "Advanced" → "Go to AI Second Brain (unsafe)".
Ini normal untuk proyek Cloud pribadi.
3. Setelah authorize, browser akan redirect ke halaman yang mungkin gagal dimuat. Itu normal. Copy URL LENGKAP dari address bar.
Bentuknya: `http://localhost:8080/?code=4/0AcXXX...`
4. Tempel HANYA nilai code (setelah code=) di sini:

Ikuti empat langkah itu. Bagian yang paling sering bikin bingung: **halaman "Google hasn't verified this app"** dan **menyalin code= dari address bar**. Diagram di bawah menjelaskan keduanya:



- 1 Klik **Advanced** (kadang tertulis "Lanjutan").
- 2 Klik **Go to AI Second Brain (unsafe)**. "Unsafe" itu wajar: ini app-mu sendiri mengakses datamu sendiri.
- 3 Setelah redirect, copy bagian yang disorot: dari sesudah **code=** sampai sebelum **&scope=**. Tempel balik ke terminal, Enter.

PENTING. Kata "unsafe" bikin banyak orang mundur. Ini aman: yang dimaksud Google adalah appmu belum lewat proses verifikasi publik mereka, padahal ini appmu sendiri yang mengakses datamu sendiri. Klik Advanced → lanjutkan.

SLOT SCREENSHOT #3: Halaman "unsafe" & copy code

Tempel di sini: foto layar peringatan "Google hasn't verified this app" (dengan tombol Advanced), dan foto address bar dengan `code=` yang harus disalin. Dua layar ini paling sering bikin peserta tersendat.

Langkah 1.7: Cek Berhasil

Jalankan lagi perintah yang sama. Kali ini dia langsung jalan tanpa minta login:

```
python3 .agent/skills/work-drive-connector/gdrive_manager.py search --query "test"
```

CEK BERHASIL. Kamu melihat daftar file dari Drive-mu (atau hasil kosong kalau tidak ada file bernama "test", itu juga berarti sukses). Kalau muncul error, tempel ke Claude.

(Opsional) Kalender. Kalau tadi kamu enable Calendar API, tes kalendermu:

```
python3 .agent/skills/google-calendar-connector/gcal_manager.py sweep --profile work --output man
```

Bagian 2: Email (Gmail)

Kabar baik: email **numpang di OAuth Google yang sudah kamu buat di Bagian 1**. Kamu tidak perlu bikin proyek atau kunci baru. Cukup pastikan Gmail API sudah nyala, lalu pakai `credentials.json` yang sama untuk connector Gmail.

Perkiraan waktu: **5-10 menit** (asal Bagian 1 sudah beres).

Langkah 2.1: Pastikan Gmail API Nyala

Balik ke console.cloud.google.com → **APIs & Services** → **Library** → cari **Gmail API** → **Enable** (kalau belum kamu nyalakan di Langkah 1.2).

Langkah 2.2: Pakai credentials.json yang Sama

Salin kunci yang sama ke folder connector Gmail:

```
cp ~/Downloads/credentials.json .agent/skills/gmail-connector/credentials.json
```

Langkah 2.3: Login Pertama untuk Gmail

Karena Gmail meminta izin tambahan (baca/kirim email), kamu login sekali lagi khusus untuk skop Gmail. Jalankan perintah `profile` untuk memicu login:

```
python3 .agent/skills/gmail-connector/gmail_manager.py profile
```

Ikuti alur login yang sama seperti Langkah 1.6 (Advanced → lanjutkan → copy `code=` → tempel). Kalau muncul layar izin yang menyebutkan akses Gmail, itu memang seharusnya.

PENTING. Kirim-email adalah aksi keluar. Sesuai guardrail second brain, AI **tidak akan mengirim email tanpa persetujuanmu**. Dia menyiapkan draft, kamu bilang kirim, baru terkirim.

CEK BERHASIL. Perintah `profile` menampilkan alamat dan info akun Gmail-mu. Untuk tes baca, jalankan `gmail_manager.py list --query "is:unread" --limit 5` dan lihat daftar email tak-terbaca muncul. Kalau macet, tempel error ke Claude.

Bagian 3: Slack

Second brain bisa menyapu channel Slack, merangkum thread, dan membalas **sebagai kamu** (bukan sebagai bot), selalu dengan persetujuanmu dulu. Untuk itu kita bikin satu "Slack App" dan mengambil tokennya.

Perkiraan waktu: **10-15 menit**.

Langkah 3.1: Pilih Jenis Token

Opsi	Token	Kelebihan	Untuk siapa
A. User Token (disarankan)	xoxp-	AI bertindak sebagai kamu, otomatis lihat channel & DM yang kamu lihat. Tidak perlu invite bot ke tiap channel.	Produktivitas pribadi
B. Bot Token	xoxb-	Jadi user bot terpisah, bagus untuk integrasi tim bersama. Harus di-invite ke tiap channel.	Integrasi tim

Untuk pemakaian second brain pribadi, pilih **Opsi A (User Token)**.

Langkah 3.2: Bikin Slack App

Buka api.slack.com/apps.

The screenshot shows the 'api.slack.com/apps' page. It features a 'Create New App' button (1), a selection menu with 'From scratch' (2) and 'From an app manifest'. Below are input fields for 'App Name' (3) with the value 'AI Second Brain' and 'Workspace' (4) with a dropdown showing 'pilih workspace-mu'. A final 'Create App' button (5) is at the bottom.

- 1 Klik **Create New App**.
- 2 Pilih **From scratch**.
- 3 Beri nama, misal **AI Second Brain**.
- 4 Pilih workspace tempat kamu kerja.
- 5 Klik **Create App**.

Langkah 3.3: Tambahkan Scopes (Izin)

Di menu kiri, buka **OAuth & Permissions**, scroll ke bagian **Scopes**. Tambahkan scope di bawah **User Token Scopes** (untuk Opsi A) atau **Bot Token Scopes** (untuk Opsi B):

```
channels:read      channels:history
groups:read        groups:history
im:read            im:history
mpim:read          mpim:history
chat:write         files:read
users:read         reactions:read
```

TIPS. Daripada mengetik 12 scope satu-satu, tempel daftar di atas ke Claude: "Ini daftar Slack scope yang harus aku tambahkan. Jelaskan singkat masing-masing buat apa, biar aku paham izin yang aku kasih." Pahami izin = setup yang aman.

(Kalau pakai Bot Token Opsi B, tambahkan juga `channels:join` dan `chat:write.public`.)

Langkah 3.4: Install & Salin Token

Scroll ke atas halaman **OAuth & Permissions**:



- 1 Scroll ke atas halaman **OAuth & Permissions**, klik **Install to Workspace**.
- 2 Klik **Allow** di layar izin.
- 3 Salin **User OAuth Token** (diawali **xoxp-**). Kalau pakai Bot Token, salin **Bot User OAuth Token** (xoxb-). Jangan bagikan token ini ke siapa pun.

SLOT SCREENSHOT #4: Salin token Slack

Tempel di sini: foto layar **OAuth & Permissions** dengan token `xoxp-` yang siap disalin. (Jangan bagikan foto ini ke siapapun. Token = kunci akun Slack-mu.)

Langkah 3.5: Simpan Token

Simpan token ke `token.env` di folder slack-connector:

```
# Opsi A (User Token):
echo "SLACK_USER_TOKEN=xoxp-token-kamu-di-sini" > .agent/skills/slack-connector/token.env

# Opsi B (Bot Token):
echo "SLACK_BOT_TOKEN=xoxb-token-kamu-di-sini" > .agent/skills/slack-connector/token.env
```

Langkah 3.6: Cek Berhasil

```
python3 .agent/skills/slack-connector/scripts/slack_client.py --action list_channels
```

CEK BERHASIL. Muncul daftar channel Slack-mu. Kalau muncul `not_in_channel` untuk channel tertentu saat pakai Bot Token, ketik `/invite @AI Second Brain` di channel itu dari Slack. Kalau pakai User Token, kamu otomatis lihat semua yang kamu lihat.

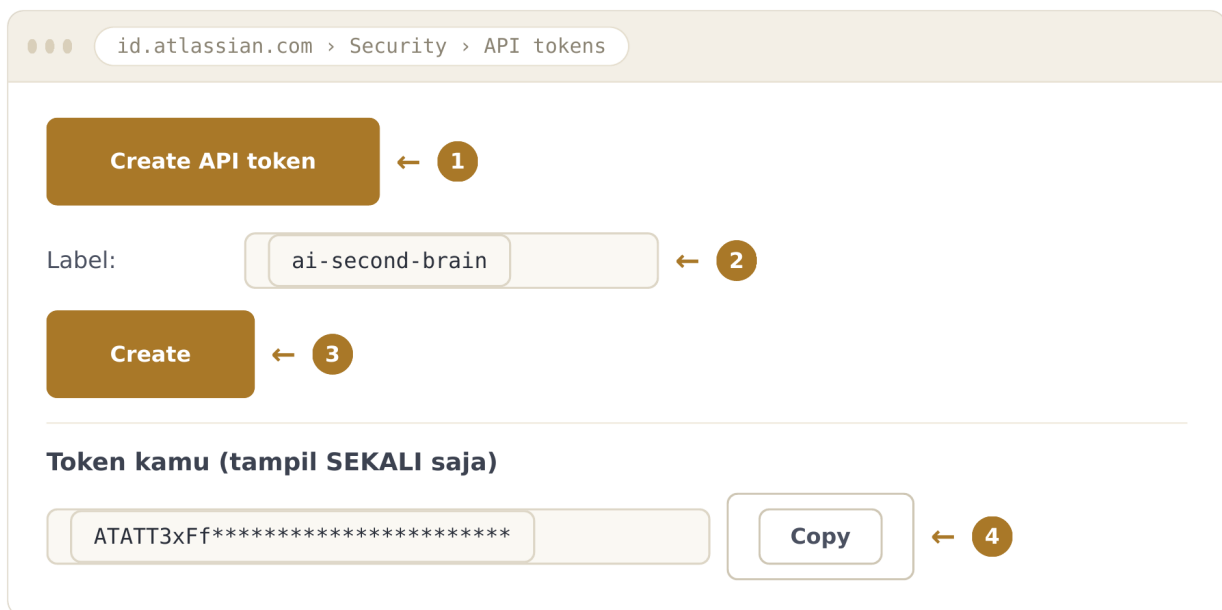
Bagian 4: Jira

Second brain bisa membaca progres sprint, mendeteksi siapa yang memegang terlalu banyak tiket, dan merangkumnya ke update harianmu. Untuk itu Jira butuh **Atlassian API token**.

Perkiraan waktu: **10 menit**.

Langkah 4.1: Bikin Atlassian API Token

Buka id.atlassian.com/manage-profile/security/api-tokens (login dengan akun Atlassian/Jira-mu).



- 1 Klik **Create API token**.
- 2 Kasih label, misal **ai-second-brain**.
- 3 Klik **Create**.
- 4 Salin token **SEKARANG**: hanya ditampilkan sekali. Kelewat? Buat token baru saja, tidak masalah.

PENTING. Token Jira hanya ditampilkan **satu kali**. Salin dan simpan langsung. Kalau kelewat, tinggal buat token baru, tidak masalah.

SLOT SCREENSHOT #5: Salin Atlassian API token

Tempel di sini: foto layar token yang baru dibuat dengan tombol **Copy**.
(Token = kunci Jira-mu, jangan dibagikan.)

Langkah 4.2: Simpan Token

Simpan email Atlassian-mu dan token ke `token.env` di folder jira-connector:

```
cat > .agent/skills/jira-connector/token.env <<'EOF'
JIRA_EMAIL=email-atlassian-kamu@perusahaan.com
JIRA_API_TOKEN=tempel-token-ATATT-kamu-di-sini
EOF
```

Langkah 4.3: Arahkan ke Situs Jira Kamu (langkah AI-Native)

Ini bagian yang beda dari connector lain, dan justru jadi latihan AI-Native terbaik. Connector Jira di repo template datang dengan **contoh** situs dan papan sprint (punya orang lain). Kamu perlu mengarahkannya ke **situs Jira dan board sprint kamu sendiri**.

Cara paling gampang: minta Claude yang lakukan. Buka terminal `claude` dari folder repo, lalu tempel:

```
Aku sudah mengisi JIRA_EMAIL dan JIRA_API_TOKEN di
.agent/skills/jira-connector/token.env.

Situs Jira-ku adalah: namaperusahaanku.atlassian.net

Tolong buka .agent/skills/jira-connector/scripts/jira_client.py,
ganti domain contoh ke situs Jira-ku, lalu bantu aku temukan board
sprint tim-ku dan sesuaikan konfigurasinya. Jelaskan tiap perubahan.
```

Claude akan membaca file, mengganti domain, dan membantumu menemukan board ID sprint yang benar. **Inilah pola AI-Native**: kamu tidak perlu ngoprek kode sendiri; kamu jelaskan tujuannya, partner AI-mu yang menyesuaikan.

TIPS. Situs Jira-mu adalah alamat yang muncul di browser saat kamu buka Jira, bentuknya `sesuatu.atlassian.net`.

Langkah 4.4: Cek Berhasil

Setelah Claude menyesuaikan domain dan board:

```
python3 .agent/skills/jira-connector/scripts/jira_client.py daily-digest
```

CEK BERHASIL. Muncul ringkasan sprint aktif dengan distribusi tiket. Kalau error soal domain atau board, balik lagi ke terminal `claude` dan tempel errornya, dia lanjutkan penyesuaiannya.

Verifikasi Semua Sekaligus

Kalau sudah pasang beberapa connector, kamu bisa tes semuanya dalam satu perintah. Ini menjalankan penyapu harian yang menyentuh banyak connector sekaligus (butuh 2-3 menit):

```
python3 .agent/scripts/daily_update_runner.py
```

Kalau ini jalan tanpa error dan menghasilkan ringkasan, artinya connector-mu hidup dan siap dipakai kerja beneran.

Kalau Macet: Peta Cepat

Gejala	Yang dilakukan
"This app isn't verified" (Google)	Wajar. Klik Advanced → Go to (unsafe) . Ini appmu sendiri.
Token expired / auth error	Jalankan ulang perintah connector itu; scriptnya otomatis minta login ulang.
"No module named X" (Python)	Jalankan <code>pip install -r requirements.txt</code> di folder repo.
Slack <code>not_in_channel</code>	Pakai User Token (<code>xoxp-</code>), atau <code>/invite</code> bot ke channel itu.
Jira: error domain/board	Balik ke <code>claude</code> , minta sesuaikan <code>jira_client.py</code> ke situs Jira-mu.
Error apapun yang tidak kamu paham	Copy, tempel ke <code>claude</code> , minta diperbaiki. Selalu langkah pertama.

Ingat prinsipnya: kamu tidak perlu paham setiap baris kode. Kamu cukup tahu tools apa yang kamu pakai, dan biarkan partner AI-mu yang menyambungkannya. Setiap error yang kamu lempar ke Claude dan dia beresin, kamu makin AI-Native.

Selamat menyambung. Setelah ini, second brain kamu punya tangan: dia bisa membaca Drive-mu, menyusun dokumen, menyapu Slack, dan membaca sprint Jira, semua tetap dengan persetujuanmu.

Panduan Koneksi AI Second Brain · dibagikan untuk peserta workshop AI Circle ·
repo template: github.com/BrianArfi/ai-second-brain